

Monitor-Assisted Quantum Error Correction in MATLAB

Can a Control Monitor Help Quantum Error Correction?

An English guide to the MATLAB teaching example. Start here; no prior surface-code knowledge is required.

The syndrome describes the qubits' check results. The monitor describes control-channel risk. A decoder combines them to choose a recovery.

What this code is for

The research question is whether a noisy observation of the control path provides useful information beyond the quantum error-correction syndrome. We compare a decoder that sees only the syndrome with one that also sees monitor alarms. Both know the same calibrated model and face the same physical errors.

The proposed hardware motivation is an optical drop or monitor port followed by a photodetector. This code starts with a simple model of the resulting risk information. It does not simulate a ring resonator, electrical waveform, microwave pulse, or quantum computer.

Run the MATLAB version

Extract the package, open MATLAB, and set Current Folder to the folder containing the .m files. No Python installation or additional MATLAB toolbox is needed.

```
run_demo
show_example
run_validation
plot_results    % Optional figure
```

Read the demo output first. Then read this guide and inspect `exact_surface_code_monitor.m`. Change one parameter at a time. All scripts and comments are in English.

Keep the scope clear

The experiment uses nine data qubits, one layer of independent X errors, and perfect checks. It sums every possible error and alarm pattern. There are no randomly sampled shots and no discarded trials.

Review status: the mathematical reference was executed and independently checked in Python. MATLAB startup was blocked in this environment, so the MATLAB edition has not been execution-verified here. The included validation script is provided for running that check in MATLAB.

1. What Does a Syndrome Measure?

It reports collective checks, rather than reading the protected quantum information.

Begin with the basic terms

Data qubit: One physical qubit used to hold the encoded state.

Logical qubit: The quantum information protected collectively by the data qubits.

Ancilla: An auxiliary qubit used to extract a check result.

Syndrome: The pattern of check outcomes, usually represented by bits.

Decoder: Classical software that uses the available evidence to choose a recovery.

A surface-code check acts jointly on a small group of neighboring data qubits, often four in the interior and fewer at a boundary. A circuit couples those qubits to an ancilla, which is then measured. The ancilla reveals the check value without directly reading the individual data-qubit states.

Z check. Joint property: A product such as $Z_1 Z_2 Z_3 Z_4$; Errors it detects: An odd overlap with X faults (bit flips).

X check. Joint property: A product such as $X_1 X_2 X_3 X_4$; Errors it detects: An odd overlap with Z faults (phase flips).

The check value is +1 or -1. Relative to an expected +1 check, we can label these as 0 and 1. A violated check constrains where an error could have occurred; it usually does not identify a unique fault.

Why time information also matters

In real surface-code operation, checks are repeated. A decoder uses which checks changed and in which round. Changes between successive outcomes are often called detection events. Their pattern in space and time helps distinguish data faults from faults in the check measurement itself.

Our small code uses one ideal set of checks. It has no ancilla-gate circuit, faulty measurements, or repeated-round history. Those are later extensions.

2. What Does Our Monitor Add?

It supplies calibrated information about control risk, rather than another qubit measurement.

The proposed physical path

An optical drop/monitor port provides a signal related to the photonic control path. A photodetector converts it to an electrical measurement. Comparing that signal with the expected response may reveal resonator detuning, optical pulse-energy deviations, or modulation-depth changes, depending on the device and calibration.

The useful connection to QEC is a calibrated estimate of the fault probability for an identifiable operation or channel. A measured optical deviation is not automatically a qubit error probability. That mapping must be established experimentally.

Syndrome: Which collective checks are inconsistent with the expected code state.

Control monitor: Which mapped control channels or operations were more likely to produce faults.

Combined evidence: Which error explanation and logical recovery are most plausible.

The abstraction used in this code

For each of nine channels, the program assumes a hidden good/bad risk state and a noisy binary alarm. The actual X fault and the alarm are independent when the risk state is fixed. Both depend on that state, so the alarm can still provide information about fault probability.

h. Meaning: Hidden control-risk state; Available to decoder?: No, except in the perfect-risk reference.

e. Meaning: Actual X-fault realization; Available to decoder?: No. Used only for probability calculation and scoring.

m. Meaning: Observable noisy alarm; Available to decoder?: Yes, for the monitor-assisted decoder.

An alarm is a noisy observation of the risk state; its interpretation depends on calibration. It does not mean an error definitely occurred. Even a perfect risk monitor does not reveal the realized fault.

A monitor can only inform noise to which its observation is sensitive. A photonic port cannot automatically diagnose every later electronics, cable, or qubit fault. The present per-channel model does not establish that a real port can provide this resolution.

3. Convert an Alarm Into a Probability

This is the step between measuring a control signal and helping a decoder.

$\text{bad_fraction} = 0.05$. Probability of a bad-risk channel.

$\text{p_good} = 0.001$. X-fault probability in the good state.

$\text{p_bad} = 0.05$. X-fault probability in the bad state.

$\text{false_positive} = 0.02$. Probability of an alarm in a good state.

$\text{false_negative} = 0.20$. Probability of no alarm in a bad state.

These numbers are teaching assumptions, not measured hardware parameters. In particular, $\text{p_bad} = 0.10$ means a 10% conditional X-fault probability, not a 10% optical amplitude error.

Without a monitor

$$p_average = 0.95 \cdot 0.001 + 0.05 \cdot 0.05 = 0.00345$$

The baseline uses this correct average probability.

With an alarm: a worked Bayes update

$$P(\text{alarm}) = 0.05 \cdot 0.80 + 0.95 \cdot 0.02 = 0.059$$

$$P(\text{bad} \mid \text{alarm}) = 0.040 / 0.059 = 0.677966$$

$$\begin{aligned} P(\text{fault} \mid \text{alarm}) &= 0.677966 \cdot 0.05 + 0.322034 \cdot 0.001 \\ &= 0.03422034 \end{aligned}$$

No alarm gives $P(\text{bad} \mid \text{no alarm}) = 0.010627$ and $P(\text{fault} \mid \text{no alarm}) = 0.00152072$.

No monitor: 0.3450%

Alarm: 3.4220%

No alarm: 0.1521%

The decoder needs $P(\text{fault} \mid \text{monitor})$, not just $P(\text{bad} \mid \text{monitor})$. The risk estimate must be converted into an actual fault-probability estimate.

4. The Nine-Qubit Code

All labels below use MATLAB numbering, starting at 1.

Top. Left: 1; Center: 2; Right: 3

Middle. Left: 4; Center: 5; Right: 6

Bottom. Left: 7; Center: 8; Right: 9

Nine data qubits and eight independent checks protect one logical qubit. The code has distance three: the shortest undetectable operation that changes logical information touches three data qubits. The present simulation adds only X faults.

1. X-type check supports: 1, 2, 4, 5; Z-type check supports: 2, 3, 5, 6
2. X-type check supports: 5, 6, 8, 9; Z-type check supports: 4, 5, 7, 8
3. X-type check supports: 2, 3; Z-type check supports: 1, 4
4. X-type check supports: 7, 8; Z-type check supports: 6, 9

Error words and syndromes

An error word is a row with nine bits. Entry 1 means an X fault occurred on that qubit; entry 0 means it did not. Each Z check records whether it overlaps the fault pattern an odd or even number of times. Two X faults in the same check cancel in its parity.

$\text{syndrome_bits} = \text{mod}(\text{error_word} * \text{HZ}', 2);$

The four syndrome bits follow the Z-check row order above. Internally, the first check is the least significant bit, giving a syndrome ID from 0 to 15. MATLAB accesses that column using $\text{syndrome_id} + 1$.

Logical operators and recovery

Logical X acts on qubits 1, 4, and 7. Logical Z acts on qubits 1, 2, and 3. An error can be corrected without identifying every physical fault: a recovery is successful if the remaining operation is a stabilizer, which leaves the encoded information unchanged.

Clearing all checks is necessary but not sufficient. A recovery can clear the syndrome while leaving a logical X error.

5. Same Syndrome, Different Recovery

A concrete example shows why extra risk information can matter.

A. X faults on: Qubit 1; Z-check syndrome: [0 0 1 0]; Logical class: 1

B. X faults on: Qubits 4 and 7; Z-check syndrome: [0 0 1 0]; Logical class: 0

Error A violates the Z check on qubits 1 and 4. Error B gives the same outcome: the Z check on 4, 5, 7, and 8 overlaps two faults, so its parity is unchanged. The syndrome alone cannot distinguish these two explanations.

They differ by logical X on qubits 1, 4, and 7. Applying recovery X on qubit 1 fixes A, but if B happened it leaves that logical X behind. Applying X on qubits 4 and 7 reverses those outcomes.

What happens when channels 4 and 7 raise alarms?

Run `show_example`. It explicitly uses the stronger-anomaly setting $p_{\text{bad}} = 0.10$. The probabilities below are the two logical-class probabilities conditioned on the available evidence.

Syndrome only. $P(\text{class } 0): 0.011830$; $P(\text{class } 1): 0.988170$; Selected recovery: X on qubit 1

Syndrome + alarms on 4 and 7. $P(\text{class } 0): 0.727551$; $P(\text{class } 1): 0.272449$; Selected recovery: X on qubits 4 and 7

The monitor changes the best logical recovery. It does not tell the decoder which error really occurred. A recovery can still fail.

Sum equivalent errors, not just these two words

For each syndrome there are two logical classes, each containing sixteen stabilizer-equivalent error words. The decoder sums every word in each class and chooses the more probable class. These probabilities use those full sums, not a comparison of only A and B.

At the default $p_{\text{bad}} = 0.05$, these same two alarms do not reverse this particular decision. Extra information need not change every recovery.

The before-recovery class is not a failure label. Failure means the chosen recovery class differs from the actual error class.

6. Follow the MATLAB Code

The main calculation stays in one function file; the other files help you run and inspect it.

`run_demo.m`: Editable parameters, result table, JSON and CSV output.

`exact_surface_code_monitor.m`: Code definition, Bayes update, exact decoder and controls.

`show_example.m`: The ambiguous syndrome and two recovery choices.

`run_validation.m`: Recovery checks, controls, independent calculation and scan.

`plot_results.m`: Optional comparison figure.

Read the main function in this order

1. `complete_parameters` fills the defaults and rejects unknown, nonfinite, nonscalar, or out-of-range inputs. Setting good and bad probabilities equal is allowed, as are all-good or all-bad risk populations. Valid numeric inputs are converted to double precision before the probability arithmetic. Converting a single value retains the value already represented at single precision.

2. `build_code` creates HX, HZ, the logical operators, and all 512 nine-bit error and monitor words. It checks commuting stabilizers, independent generators, the logical relationships, distance three, and the size of each logical class.

3. `joint_probabilities` calculates each error word's probability by multiplying nine independent fault/no-fault probabilities. It sums those weights into a 2-by-16 table: logical class by syndrome.

4. `evaluate_sensor` uses Bayes' rule to calculate fault probabilities for alarmed and quiet channels. For every monitor word, it constructs the table, chooses the most likely class for each syndrome, and adds the probability mass in the other class.

5. The main function compares syndrome-only, noisy monitor, independent-record, perfect-risk, and uninformative-monitor decoders. Each comparison scores the same physical distribution.

How to change one setting

```
result = exact_surface_code_monitor(struct('p_bad', 0.10));  
result = exact_surface_code_monitor( ...  
    struct('false_positive', 0.005));
```

Changing `p_bad` changes the underlying noise. Changing only `false_positive` changes monitor quality while keeping the physical error distribution fixed.

7. Read the Results Honestly

Reference values from the checked exact mathematical model.

Syndrome only: 0.0002113845683

Syndrome + noisy monitor: 0.0002099323731

Independent monitor record: 0.0002164980031

Perfect risk monitor: 0.0001538930539

Uninformative monitor: 0.0002113845683

The default relative reduction is 0.687%, about 1.45 fewer expected logical failures per million modeled experiments. This is a small benefit, not a 0.687 percentage-point decrease.

Setting only `p_bad` to 0.10 gives baseline 0.0006226238870 and monitored probability 0.0004820882069: a 22.57% relative reduction. Both decoders now face worse noise. The larger relative gain does not mean the physical system became more reliable.

Results are exact finite probability sums, up to floating-point precision. Their precision does not improve by increasing a shot count.

8. Validation and Scope

Separate correct arithmetic, credible assumptions, and hardware evidence.

What `run_validation` checks

The script tests all 512 X-error words and both possible recovery classes. It verifies that a recovery clears the syndrome, and that the residual is a stabilizer exactly when recovery succeeds. It also checks single-X-error correction and the original default and stronger-anomaly numerical values. It also rejects invalid inputs and checks that accepted single and integer inputs give the same results as their explicit double conversions.

The independent check constructs the full 512-by-512 error/monitor joint table by summing over hidden good/bad states, without using the main function's Bayes formulas. It compares the baseline, monitor, and independent-record probabilities at two parameter settings. All twelve predefined scan points are saved, including points with little improvement.

No noise: No logical failures.

`p_good` equals `p_bad`: Risk alarms cannot distinguish fault probabilities.

Uninformative sensor: A calibrated decoder should match the baseline.

Perfect risk sensor: Best risk information; still not the realized fault.

Independent monitor record: The association with this experiment is broken; decisions may get worse.

Execution status: all five MATLAB files passed MATLAB R2024b Code Analyzer without diagnostics. The Python mathematical reference passed its checks. This MATLAB port includes equivalent validation but could not be executed in the current environment because MATLAB startup/access was blocked. Do not label its validation report as passed until you run `run_validation` successfully.

The new model still does not include optical physics, noisy check circuits, multiple rounds, coherent or correlated noise, real monitor calibration, or a real-time latency requirement. A real drop port has not yet been shown to provide the assumed channel-risk information.

9. A Simple Student Work Plan

Understand and reproduce this model before adding another simulator or more physics.

First pass

1. Run `run_demo`. Explain `baseline_ler`, `monitor_ler`, and `relative_reduction` in your own words. Identify which parameters are assumptions.
2. Calculate the syndrome for X on qubit 1 and X on qubits 4 and 7 by hand. Explain why the two recoveries cannot both be correct for the same actual error.

3. Recalculate $P(\text{alarm})$, $P(\text{bad} \mid \text{alarm})$, and $P(\text{fault} \mid \text{alarm})$. Explain why the last probability is the one used by the decoder.

Second pass

4. Run `show_example` and `run_validation`. Check the default and stronger-setting results against the reference values, and inspect all twelve saved scan points. Investigate any failed check before reporting an improvement.

5. Change one parameter at a time. Try $p_{\text{bad}} = 0.02, 0.05, \text{ and } 0.10$. Then hold physical noise fixed and vary `false_positive`. Report the baseline and monitored probabilities together, including cases with no useful gain.

6. Explain each control: no noise, equal risk-state error rates, an uninformative monitor, a perfect risk monitor, and an independent monitor record. Do not discard experiments with alarms.

Only then extend the project

A sensible next step is repeated noisy syndrome extraction and circuit-level decoding. Another is calibrating a measured monitor signal against identifiable control deviations. For sampled experiments, compare decoders on the same trials and report uncertainty in their paired difference. Monitor records must not reveal true injected fault labels.

Primary references

Fowler et al., Surface codes: Towards practical large-scale quantum computation (2012). <https://arxiv.org/abs/1208.0928>

Pattison et al., Improved quantum error correction using soft information (2021). <https://arxiv.org/abs/2107.13589>. This discusses richer measurement information, not our optical control monitor.

For optional later tools: <https://github.com/quantumlib/Stim> and <https://pymatching.readthedocs.io/en/stable/>. They are not dependencies of this MATLAB starter.